Introdução à Lógica e Programação

Depuração

Depuração

É o processo sistemático para encontrar, analisar e corrigir erros (*bugs*) em um programa.

É essencial para entender o fluxo de execução do seu código e identificar a raiz de problemas.

Em inglês, é conhecido como *debug*.

Erro de sintaxe

Um erro de sintaxe ocorre quando o programador fere alguma das regras de codificação da linguagem de programação (e.g., aspas não fechadas, indentação indevida, falta do símbolo de dois pontos após determinadas instruções, etc).

Erros de sintaxe são detectados facilmente pelo compilador ou interpretador da linguagem de programação.

```
x = int(input())
if x % 2 == 0
  print('Par')
else:
  print('Ímpar')
```

Exercício 1

Corrija o programa abaixo eliminando o erro de sintaxe.

```
nome = input()
if nome == "Paulo":
print("Oi, Paulo")
else:
  print("Você não está registrado")
```

Erro de execução

Um erro de execução ocorre em algum ponto da execução do programa. A sua causa não pode ser antecipada pelo compilador ou interpretador (e.g., referência a uma variável inexistente, operação indevida, acesso a um arquivo inexistente, etc).

A linha de código que gera o erro é facilmente identificada pelo interpretador Python.

```
x = int(input())
if y % 2 == 0:
  print('Par')
else:
  print('Ímpar')
```

Exercício 2

Modifique o programa abaixo evitando que ocorra um erro de execução quando o usuário informar um divisor com valor zero.

```
a = int(input('Digite o dividendo: '))
b = int(input('Digite o divisor: '))
q = a / b
print('Quociente:', q)
```

Erro de programação

É um erro de lógica ou de cálculo que faz com que o programa produza resultados incorretos, ou produza um erro em tempo de execução.

São os erros mais difíceis de serem identificados pois não são reconhecidos pelo compilador ou interpretador. A sua identificação se faz através da análise de código ou pelo processo de depuração.

```
a = int(input('Digite um inteiro: '))
quadrado = a ** 2
cubo = a ** 2
print(f'Quadrado de {a}: {quadrado}')
print(f'Cubo de {a}: {cubo}')
```

Depurador

É uma ferramenta que permite executar o código passo a passo, linha por linha.

Também permite a inspeção e verificação do valor de variáveis em tempo de execução.

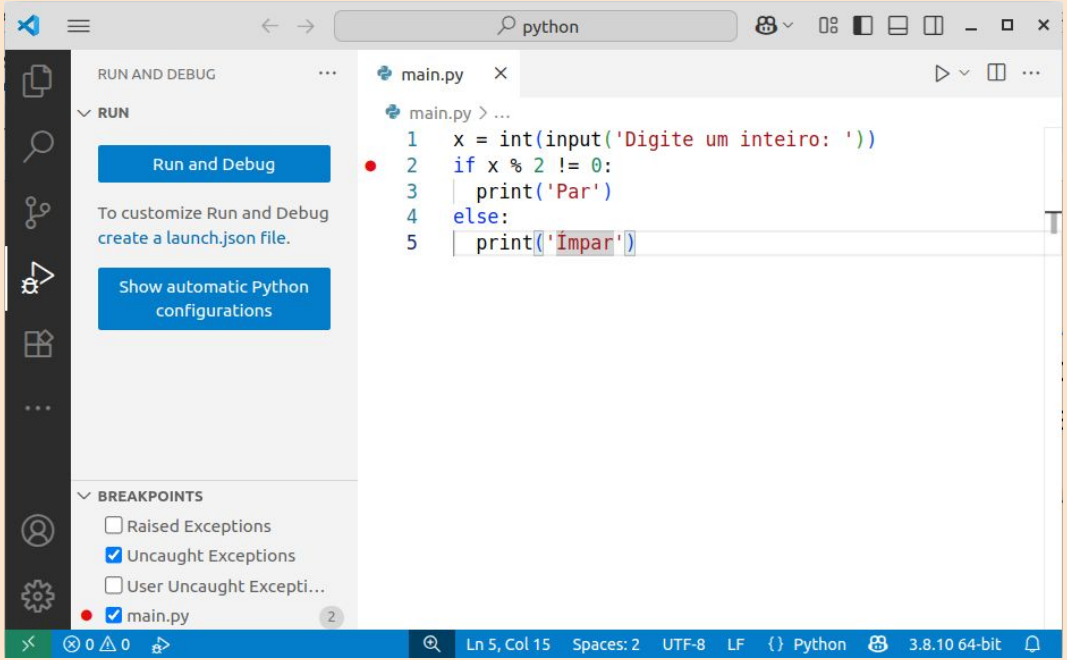
Facilita a identificação dos trechos de código que contém erros de programação.

A extensão para Python do VS Code inclui um depurador. Para iniciá-lo, use o opção de menu **Run** > **Start Debugging** ou tecle **F5**. Em seguida, selecione a opção **Python File Debug the currently active Python** file na caixa de diálogo que surge.

Usando o depurador

Breakpoints são pontos de parada que podemos inserir ao longo do programa. O depurador pausa a execução do programa ao encontrar um breakpoint.

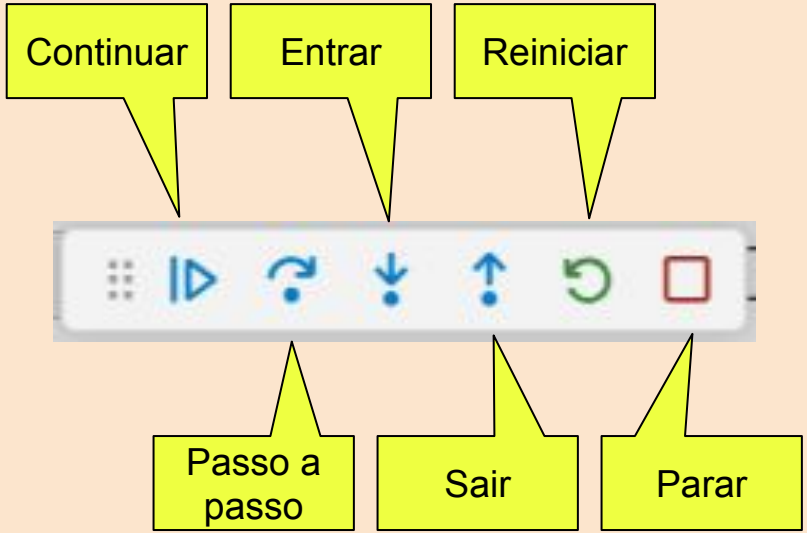
Basta clicar à esquerda do número de linha para inserir um breakpoint, o qual é assinalado por um círculo vermelho.



Usando o depurador

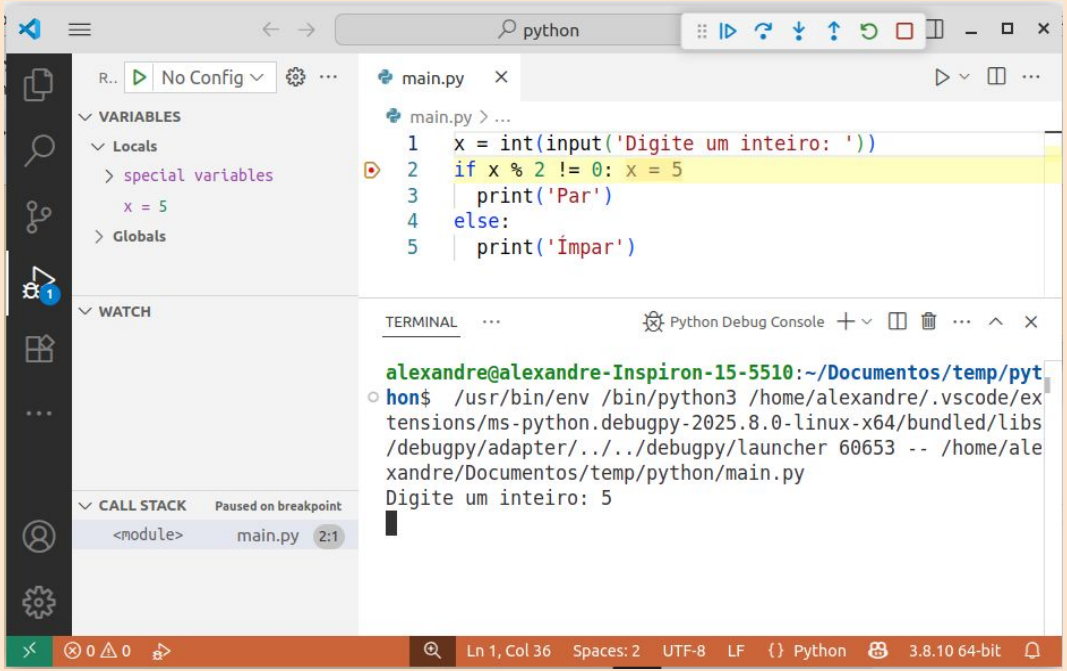
A barra de ferramentas de depuração aparece quando a depuração está ativa. Ela contém os seguintes controles:

- **Continuar** (F5): Continua a execução do código até o próximo breakpoint ou o fim do programa.
- **Passo a Passo** (F10): Executa a próxima linha de código, pulando chamadas de função (não entra na função).
- **Entrar** (F11): Executa a próxima linha de código e entra em chamadas de função.
- **Sair** (Shift+F11): Sai da função atualmente em execução e retorna para a linha de onde ela foi chamada.
- **Reiniciar** (Ctrl+Shift+F5): Encerra a sessão de depuração atual e a reinicia do começo.
- **Parar** (Shift+F5): Encerra completamente a sessão de depuração.



Usando o depurador

A visão lateral **Run and Debug** contém o painel **VARIABLES**, o qual permite a visualização das variáveis com seus respectivos valores.



Exercício 3

Corrija o programa abaixo de forma que ele imprima a mensagem correta de acordo com a sua entrada. Comece identificando em que situações o programa apresenta erro.

```
n = int(input('Digite um inteiro: '))
if n < 5:
  print('A entrada é maior do que cinco')
else:
  print('A entrada é menor ou igual a cinco')
```

Exercício 4

Corrija o programa abaixo de forma que ele imprima a mensagem correta de acordo com a sua entrada. Primeiro identifique em que situações o programa apresenta erro. Em seguida, use o depurador para descobrir o erro de código.

```
# Programa que informa se a entrada é um múltiplo de 3 e/ou de 5
n = int(input('Entrada: '))
if n % 3 == 0:
  print(f'{n} é múltiplo de 3')
elif n % 5 == 0:
  print(f'{n} é múltiplo de 5')
elif n % 3 == 0 and n % 5 == 0:
  print(f'{n} é múltiplo de 3 e de 5')
else:
  print(f'{n} não é múltiplo nem de 3 nem de 5')
```

Exercício 5

Corrija o programa abaixo de forma que ele imprima a mensagem correta de acordo com a sua entrada.

```
turno = int(input('Entrada: '))
if turno == 'V' and turno == 'v':
  print('Vespertino')
elif turno == 'M' and turno == 'm':
  print('Matutino')
else:
  print('Valor inválido')
```

Exercício 6

Corrija o programa a seguir de forma que ele sirva como solução do problema apresentado.

Problema: Escreva um programa que leia a idade de uma pessoa e classifique-a em uma das seguintes categorias: Criança (0 a 12 anos); Adolescente (13 a 17 anos); Adulto (18 a 59 anos); Idoso (60 anos ou mais).

```
idade = int(input("Digite sua idade: "))
if idade <= 12:
  print("Criança")
if idade <= 17:
  print("Adolescente")
if idade <= 59:
  print("Adulto")
else:
  print("Idoso")
```

Problema: Cálculo do Salário Líquido

Um funcionário recebe um salário bruto mensal. Sobre esse valor, são aplicados os seguintes descontos:

1. INSS: 11% do salário bruto
2. Imposto de Renda (IR):
    a. Se o salário bruto for até R$ 2.000, isento
    b. De R$ 2.000,01 até R$ 5.000, desconto de 15%
    c. Acima de R$ 5.000, desconto de 27,5%

Escreva um programa que leia o salário bruto e calcule o salário líquido, aplicando os descontos na ordem correta: primeiro o INSS, depois o IR sobre o valor restante.

<u>Exemplo de entrada e saída esperada</u>

Entrada (salário bruto): R$ 6.000
Cálculo esperado:

- INSS: 11% de 6.000 = 660,00
- Salário após INSS: 5.340
- IR: 27,5% de 5.340 = 1.468,50
- Salário líquido: 5.340 - 1468,50 = 3.871,50

Saída esperada: Salário líquido: R$ 3.871,50

Exemplos			
Salário Bruto	**INSS**	**IR**	**Salário Líquido**
6.000	660,00	1.468,50	3.871,50
1.800	198,00	0,00	1.602,00
3.000	330,00	400,50	2.269,50

Exercício 7

Corrija o programa a seguir de forma que ele sirva como solução do problema do cálculo do salário líquido.

```
salario_bruto = float(input("Digite o salário bruto: "))

inss = salario_bruto * 0.11
if salario_bruto <= 2000:
  ir = 0
elif salario_bruto <= 5000:
  ir = salario_bruto * 0.15
else:
  ir = salario_bruto * 0.275

salario_liquido = salario_bruto - inss - ir
print(f"Salário líquido: R$ {salario_liquido:.2f}")
```

Exercício 8

Corrija o programa a seguir de forma que ele sirva como solução do problema do cálculo do salário líquido.

```
salario_bruto = float(input("Digite o salário bruto: "))

inss = salario_bruto * 0.11
salario_pos_inss = salario_bruto - inss

if salario_bruto <= 2000:
  ir = 0
elif salario_bruto <= 5000:
  ir = salario_pos_inss * 0.10
else:
  ir = salario_pos_inss * 0.20

salario_liquido = salario_pos_inss - ir
print(f"Salário líquido: R$ {salario_liquido:.2f}")
```

Exercício 9

Corrija o programa abaixo de forma que ele produza as saídas corretas.

```
# Programa que calcula as raízes de uma equação do 2º grau
import math
a = float(input('Coeficiente de x ao quadrado: '))
b = float(input('Coeficiente de x: '))
c = float(input('Coeficiente livre: '))
delta = b ** 2 - 4 * a * b
if delta < 0:
  print('A equação não possui raízes reais.')
elif delta == 0:
  x = -b / 2 * a
  print(f'Raíz: {x:.4f}')
else:
  x1 = -b + math.sqrt(delta) / 2 * a
  x2 = -b - math.sqrt(delta) / 2 * a
  print(f'Raiz 1: {x1:.4f}')
  print(f'Raiz 2: {x1:.4f}')
```

Informações bibliográficas

Autor: Alexandre Gomes de Lima

Data: junho de 2025.

Direitos autorais: CC BY 4.0
(https://creativecommons.org/licenses/by/4.0/deed.pt-br)